

Active Inference ModelStream 007.1 ~ Conor Heins & Daphne Demekas ~ p

ModelStream | Conor Heins & Daphne Demekas ~ pymdp | 1:47:29

Hello and welcome everyone.

This is the Active Inference Institute.

It's November 15th, 2022, and we're in Model Stream 7.1.

We're going to be discussing PyMDP, a Python package for Active Inference in discrete state spaces.

We will all say hello, then we'll pass to Connor for a presentation.

Following the presentation, we'll have some discussion, take a look over PyMDP scripts, take any questions that are coming up in the live chat.

So thanks to the authors for joining today and also to Jakob.

We'll just start by saying hello.

So I'm Daniel.

I'm a researcher in California, and I'm really excited to learn a little bit more about PyMDP and see how Active Inference gets applied.

And I'll pass to Jakob.

Hi everyone, I'm Jakob.

I'm a student in the UK and also very excited to hear more about PyMDP and discuss the recent developments and plans for future development.

I'll pass it to Daphne.

Hi, I'm Daphne.

I'm working here in London.

I used PyMDP a lot for work that I did with Connor on my master's thesis and also just work that we've been doing since then.

So I definitely think it's a really, really great package.

And I'm happy that people are going to start using it more.

Excellent.

All right, Connor.

Thanks a lot for joining.

Take it away.

Great.

Thank you.

Thanks for the invite.

I'm glad that we arranged this.

It's like a nice opportunity to go on the live stream.

I've been on a few times now, so it's always nice to come back.

So I don't have a slide introducing myself, so I'll just say a quick sentence just about who I am.

So I'm Connor.

I'm a PhD student in biology at the Max Planck Institute for Animal Behavior in Constance in Germany.

And most of my work is about applying active inference and kind of the Bayesian lens on cognitive science and complex systems, applying that to collective behavior, like collective animal behavior.

But today I'll be talking about kind of one of my side PhD projects, which has been developing this PyMDP package, which was very much a collaborative group effort with a bunch of people from the active inference community.

So yeah, let's dive in.

So yeah, the paper, the package has actually been out for a while, and the paper came out earlier this year.

And it's just, I want to begin by emphasizing that PyMDP is very much a work in progress.

Even though we released a paper and it's definitely like a usable standalone package, there's always a ton to keep developing.

And towards the end of the presentation, I'll talk about some of those ongoing developments, which are really exciting in my opinion, and will really open up the usage and extendability of the package.

So basically, I mean, this is an active inference institute podcast or live stream.

So I don't have to spend too much time motivating, I don't think, active inference.

But in short, the PyMDP package is a Python package for simulating and running active inference processes in discrete state spaces.

And the discrete state space case is a very well studied case, and very well characterized, and it's been very popular in like models of decision making, like discrete decision making and planning, and has seen a lot of application in the neurosciences as models of like discrete decision making behavior in, for instance, humans or other animals.

So just I'll give a little outline of the presentation.

So first, I'll just introduce the team of people who have worked on PyMDP and were authors on the paper.

But the actual effective team goes much larger than that, because there's a lot of people who are developing or using it and contributing in their own ways that weren't actually co-authors on the paper.

And then I'll discuss the motivation for the package.

I'll give a brief overview of active inference in discrete state spaces.

But as I mentioned, I think this is a really nice like venue to have this discussion about PyMDP, because I don't think I need to go too deep into discussing what active inference is.

So that'll save us some time to like get more into the depth of PyMDP.

I'll talk about the existing approaches to simulating active inference agents, like namely in MATLAB using SPM and just kind of compare and contrast PyMDP with SPM and help us understand like kind of where PyMDP is coming from in terms of its origins in SPM, really.

So then we'll talk about some of the features of PyMDP and its general package structure.

And then we'll dive into a few usage examples where I'll show some like outputs of simulated agent behavior and then the code that accompanies that just to kind of demonstrate what the general flow of PyMDP looks like.

And then at the end, I'll just discuss some of the future directions and ongoing active branches, I guess, of PyMDP, their ongoing development efforts that I think will make PyMDP super exciting and extendable to all kinds of new use cases.

So I'm really excited about that.

And as Daniel was saying, if at any point someone wants to butt in or has questions for clarification, just feel free to let me know and we can kind of dwell on some points for longer.

Okay, so I'll start by introducing the team.

So the co-authors in the paper in addition to me were Barron Millage, Daphne Damekas, who is here today, Brennan Klein, Carl Friston, Ian Cousin, and Alexander Chance.

So Carl and Ian are my co-PhD supervisors.

And Carl is the original progenitor of the MDP or discrete state space formulation of active inference in MATLAB.

So it's nice to kind of have his stamp of approval on our work here and his kind of co-sign on that.

And then just I think it's important to emphasize how critical everyone here is to the package.

And it wasn't really just I did do, I guess, most of the actual software development.

But the early stages of PymDP were really conversations between me, Brennan, and Alec back in 2019 or maybe even earlier,

just about the need for a Python package that does active inference.

And I think probably other people were having similar conversations around that time.

But it was because we all kind of came together that we're able to build this thing in not too long of a time.

I mean, more than two years.

And it could have probably been done faster if I was working full time on it.

But it was really fun to kind of watch the progression of this.

And then as Daphne said, she's actually one of the first people who really used active PymDP in her own master's thesis.

And not only that, but in a very ambitious application, which is like multi-agent collective behavior, having a bunch of PymDP agents interacting with each other to simulate the kind of opinion dynamics and echo chambers and social networks, which is really exciting.

So that was really gratifying to work with her on that.

And yeah, so it's really nice also that Daphne is part of this because it's like a very good example of a pioneer in the PymDP active inference community.

And then Barron also, me, Alec, and Barron did a lot of work on developing some of the more sophisticated

message passing techniques in active inference in PymDP.

And Barron was also really critical in helping me write the paper.

And he's just great at writing and conceptualization and just was also used actually PymDP in some of his own work on successor representations in active inference, which is kind of cool.

Okay, so motivation.

One of the biggest things which everyone here is, I'm well aware of, is there's just much more popularity and interest in active inference these days in the past 10 years and especially the last five years.

So it's kind of obvious that we need some more general user-friendly frameworks for actually letting people learn about active inference, first of all, from a pedagogical perspective, as well as applying it in their own research or industrial applications or whatever they want to actually do with active inference.

And there's a lot especially of interest in active inference from certain communities, not just neuroscience anymore, but things like machine learning, data science, engineering, network science, even software development, software engineers.

And a lot of those interested fields, the most dominant language is Python.

So when a lot of times what I've heard is when people come to learn about active inference and they have a background in Python or R or something and they figure out that it's all in MATLAB, there's kind of a barrier to entry because they either don't know MATLAB or they have a hard time parsing MATLAB code, especially if they're from like non-array programming frameworks.

Like maybe they're a front end web developer who uses JavaScript or something and they're not actually familiar with like big multi-dimensional array programming.

So that was another motivation for making specifically a Python package that does active inference.

And then finally, because it's in Python, that means that we're now creating an ecosystem that can talk to other ecosystems.

So ideally, Pymdp isn't going to be just used in a script that only uses Pymdp.

It's going to be used with other software packages, some of them having to do with artificial intelligence or network science or all kinds of frameworks that are relevant.

And so you can now kind of plug and play with Pymdp agents and put them in environments like OpenAI Gym for reinforcement learning, for instance.

And now you can work with active inference in diverse applications.

And that's really great for Python because Python, by design and by its community-driven development, just has so many different packages that were built very well for some specific thing.

So now that it's in Python, you can kind of use it in tandem with all those other Python packages.

Okay, so now a brief intro to active inference.

So the fundamental paradigm of active inference is that you consider an agent embedded in its environment.

And unlike kind of traditional, more passive approaches to perception and behavior where you kind of consider the environment gives you information, you do some sensory motor transformation, and then you perform an action.

Active inference very much emphasizes the fact that inference or the problem of dealing with uncertainty characterizes both perception or what we call like state estimation or belief updating, as well as action, which is where the active inference part comes into play.

So agents are not only updating their beliefs about the states of the world, the hidden states out there, by minimizing this bound on surprise called free energy.

And that's where these kind of Helmholtzian ideas of perception as inference come from.

But also you're minimizing surprise or a bound on surprise also to choose your actions.

So inferring actions becomes just another sort of inference problem.

So policies or sequences of actions are considered latent variables or hidden states.

And then you also do inference about those.

And by casting both sides of the perception action coin as an example of surprise minimization, what you events are agents that kind of display purposeful and curious behavior.

And there's very, like, there's nothing about active inference that means you have to use discrete state spaces or PIMDP.

That's just one particular type of or class of generative models for active inference.

But the PIMDP discrete state space generative models are really easy to work with active inference because a lot of these quantities that I'm showing here are very easy to compute when you're dealing with PIMDPs or partially observed Markov decision processes.

So we'll get into all the mathematics in a little bit.

But that's just the basic paradigm of active inference.

Agent and environment trying to minimize surprise, both doing perception and action by minimizing surprise.

And just for a more in-depth mathematical review of active inference in discrete state spaces, namely using these partially observed Markov decision processes, I would recommend reading this paper, which is excellent in the Journal of Mathematical Psychology by Lance, Thomas, Noor, Sebastian, Victorita, and Carl.

It's just a really great description from a formal basis of how we get to the update equations for active inference, starting from the most, like, formal treatment of categorical and Dirichlet distributions.

And also the PIMDP paper, the version we have on archive also has a bunch of appendices that do a lot of the similar sort of math.

So I would also refer people to that.

Okay, so now let's get into the generative models that form the bread and butter of the agent's brains in PIMDP, essentially.

So central to active inference is writing down a generative model, which is just a specification of how an agent believes its world works.

How does its environment, how does it believe its environment influences itself, like the dynamics of the world progress?

And how does that, those hidden state dynamics also give rise to observations?

That's all encoded in what we call a generative model or a world model, some people also call it.

So in PIMDP, we only deal with a very specific sort of generative model, which are called these partially observed Markov decision processes.

So these are a classic model of sequential decision making and planning under uncertainty.

They're not unique to active inference.

People use POMDP models for classical reinforcement learning and all kinds of decision making problems.

They're called Markovian, Markov decision process, because the state at the current time only depends on

the action in the state at the previous time.

So that's the definition of a Markov process.

They kind of have this shallow temporal dependence that, for instance, a non-Markovian process doesn't have, like a process that has longer term or deeper temporal dependencies that stretch further back in the past.

POMDPs are often but not always formula in discrete state space and discrete time.

There's nothing about the word POMDP that means they have to be these multinomial or categorical distributions.

But just when we talk about POMDPs and active inference, we're almost always talking about these discrete ones that have to do with, basically, you can only be in one of K discrete states at a time and you only progress to one of K discrete states at the next time.

So everything is discrete, but there's nothing intrinsically about these Markov decision processes that has to be discrete.

I think it's just worth mentioning that because that's an important kind of conflation that people often make when they see the word POMDP.

And the reason we decided to use this discrete POMDP generative model for active inference is not just because of the applications to sequential decision making and planning, but also there's just a massive pre-existing active inference literature since 2010, 2011 on using POMDPs as generative models for decision making tests.

So all the mathematics for doing active inference with these models is already done.

So we didn't have to invent any new math or theory to actually code this up because a lot of it has already been written in papers for like 10 years.

So that made our life easier when developing it.

So now let's dive into just the main components of these POMDPs.

I'm only listing the four major ones here, but there's other components that we can discuss if anyone's interested.

But essentially this line at the top is a description of the generative model in terms of a joint distribution over hidden states S and observations O into the future.

So because of the Markovian nature of this generative model, you can write the joint distribution with this factorized basically product of priors and likelihoods that factorizes across time.

So that's why there are those products over time in the top, but it doesn't matter understanding the math right now.

But the main components, which we'll map onto a schematic in a second, are the agent's beliefs about how the hidden states cause observations, which we encode in something called the observation model or the likelihood mapping, often called the A matrix or the A array.

So this is a probabilistic representation of how do hidden states at the current time affect or give rise to observations at the current time.

Second, we have the transition or dynamics model, which is another sort of likelihood that encodes the agent's beliefs about how hidden states at one time relate to hidden states at the next time.

So this maps, this is what the agent uses to make forward predictions about how will the world evolve if this happens or if that happens, as well as to carry kind of messages or empirical priors from the past.

Given where I was yesterday, where must I be now, given my beliefs about how the world evolves in time.

That's all encoded in the B array or the B matrix.

And then these final two are kind of priors.

The first of which is very important, which is called the C array or the C vector, which encodes the agent's prior beliefs about what observations it's likely to encounter.

And as we said in the beginning, active inference is all about casting both action and perception as an inference problem.

So active inference as a framework kind of turns the classic paradigm of reward functions and reinforcement learning on its head by saying, instead of a reward function, just equip the agent with a kind of optimistic belief that in the future, I will see this kind of data.

And then by performing inference with respect to a generative model that has that prior belief in it, the agent will look like it's searching for observations that conform with its priors.

So this goes in line with this idea that active inference is a process of self-fulfilling prophecy.

The agent believes I'm more or less likely to see certain observations.

And then by doing inference about policies with such a bias to generative model, the agent will kind of bring itself to actually realize its prior preferences or these prior beliefs about observations.

So that's all encoded in the C vector.

The first time step of the simulation.

So this isn't really a necessary thing in active inference, but oftentimes when we're simulating agents, we use a finite temporal horizon with a start time and an end time.

So if you have a finite temporal horizon, it means that you have to basically plug in a prior belief about what the world looks like at time step one.

And that prior belief is encoded in this D vector.

And we can just intuitively sketch out the POMDP generative model as a Bayesian graph where nodes are connected by edges, by these arrows if they depend on each other.

So these red nodes represent the hidden states transitioning to each other over time.

And the blue nodes, the O nodes represent the agent's beliefs about how observations are generated from those hidden states.

So that's just a graphical representation of the POMDP.

And the A matrix encodes, as we said, the agent's beliefs about how those red nodes give rise to the blue nodes at any time.

So usually we assume that this A matrix is time invariant.

So they believe that this observation mapping doesn't itself depend on time.

That's at least the assumption made in most POMDPs.

And then there's also the agent's time invariant beliefs about how the transition dynamics of the world bring states at time t minus one to states at time t .

And then policy selection or action comes into play in active inference by framing actions as controlled transitions.

So the B matrix does not just say how should the world look now given how it was yesterday, but how should it look now given how it was yesterday and the fact that I took this action at time t minus one.

So the B matrix is not only conditioned on past states, but also on previous actions.

So policies π_i come into play as a belief or a distribution over those actions.

So the actions directly influence the transitions and then the transitions affect the states.

So that's how kind of we think about action in a lot of POMDP scenarios.

And actions or policies π_i are sequences of actions or collections of actions.

And then we have this critical objective function called the expected free energy that determines which actions are more likely than others.

So by minimizing expected free energy, we optimize a belief about policies.

And then the expected free energy itself is a function of your generative model, your beliefs about the world, which includes this biased prior belief about what observations you expect yourself to see.

So the kind of reward function enters policy selection via this expected free energy.

And that's why we're able to still kind of call all of this a form of an inference problem because we're minimizing this expected bound on surprise.

So and in doing so, we kind of infer distribution of our policies, which we then sample from to actually generate actions and change the world.

And then as I said, the D vector is basically just a prior on that initial hidden state distribution.

So yeah, in summary, to build a POMDP active inference model, you have to write down or encode these A, B, Cs, and Ds.

And there's some other priors too that we can talk about, like a prior over policies, which is called an E vector.

But for now, you can just think of most of the heavy lifting in active inference as consisting in writing these things down, actually encoding what your agent believes about the world that it exists in.

And these things in the case of these categorical discrete distributions, they end up looking like matrices and vectors.

Since everything is categorical distributions, you're not dealing with continuous infinite dimensional spaces.

You're dealing with things that have a discrete number of entries, like a 4x4 matrix or 5x5 matrix.

Yeah, I think before we...

Yeah, so the next part I'm going to talk about, like the MATLAB Python dialectic.

Before we move on to that, though, should we maybe pause if there's any questions?

Yeah, great.

Thank you.

Daphne or Jakob, do you want to provide any thoughts or reflections?

There's one thing that I've always been a little bit confused about, which might be nice to resolve now, which is in the parameterization of the prior preferences.

Can you explain, like, why exactly it is that when you initialize them, you just do them with, like, integers greater than or equal to 1, but then you, like, softmax them later on when you actually use an inference?

Like, why do we not define our prior preferences as probabilities?

Yeah, that's a good point and a good thing to mention.

So, usually in both the MATLAB and the Python implementation, so when building PyMDP, we made the decision to do the same thing.

When you're encoding that C vector, instead of encoding it as probabilities, which it ultimately should be, what you really write down is the log of the C vector.

So, you write it down in terms of relative log probabilities.

And the reason that's potentially a more intuitive parameterization rather than writing it down directly in terms of probabilities is because the log of the probability is more like the actual reward.

So, if we go back to this generative model, the expected free energy being a free energy is actually, and a free energy is sort of a kind of KL divergence.

It's encoded in terms of bits, which is naturally in a logarithmic space.

So, if you're writing down like a prior in terms of in log space, you know kind of if I change the number of log units in my prior preferences, it's more linearly related to the change in the expected free energy for a policy that leads to that observation.

Whereas if you're writing down the priors in terms of probability space, the resulting change in the expected free energy from changing something in probability space is not going to be linear.

It's going to be a nonlinear change.

So, if we think of rewards as log probabilities, something that's like one extra log unit or one natural log, one nat, or one bit more valuable than another thing, that will actually reflect itself in terms of the expected free energy difference between seeing thing one versus thing two.

Whereas if we encode those two things in terms of probabilities, that expected free energy difference won't be as intuitive simply because it won't be linear.

So, that's like a quick reason for why we decide to encode things in log space.

But there's nothing mathematically necessary by any means.

You could easily just write the priors directly in terms of probabilities.

It's more of a, it's like a, it's just a way to make it more reward-like, really.

Yeah, that makes a lot of sense.

Thanks.

Thanks, Jakob.

Yeah, I'm wondering.

Is there a requirement on the A and B matrices or tensors to be actually encoded as categorical matrices?

Or, because thinking about like the function of the A matrix and the way the agent uses it to infer its hidden state from an observation,

I'm thinking whether it would be possible to encode high-dimensional state spaces in something like a neural network that can approximate these probabilistic representations as well,

and learn the representation, the relationship between observations and hidden states.

Yeah, definitely.

Yeah, definitely.

Yeah, so basically...

Sorry.

There might be a like, but yeah, basically whether you think that would be possible.

Because I feel like the Python implementation, which I'm sure you'll get into the advantages of Python over MATLAB,

also affords the interoperability with other libraries and within data science.

Totally, yeah.

Mm-hmm.

Totally, yeah.

That's a great point.

So I kind of mentioned towards the beginning when we talked about POMDPs, there's nothing about POMDPs that means all these things have to be matrices.

The main thing that defines a POMDP is that it's partially observable so that the agent only gets to see the blue nodes,

and that it's a Markovian process so that there's like a temporal shallowness in the memory of the hidden states.

But you could replace A and B with any kind of parameterized...

It has to be a proper likelihood function, so it has to have the properties of being a likelihood.

There's nothing that means that it can't be a multivariate Gaussian or a Cauchy distribution or a Bernoulli distribution or...

Yeah, any kind of exponential family or some like parametrized neural network like that people use oftentimes in reinforcement learning.

Like they'll parametrize a dynamics model just with a bunch of neural networks that say take time...

State at time t and move them to state at time t plus one.

The reason there's difficulties with that is simply because it's unclear how to compute things like the expected free energy once you start moving into these not as nicely behaved distributions.

So a lot of the reason that this framework was developed in the discrete categorical state version is not only because for modeling low dimensional tasks like someone playing a slot machine or deciding to go left or right in a Y maze.

It's easy to use these discrete distributions to kind of give a good description of that behavior.

That's one thing. But also is literally a mathematical tractability reason.

There aren't always closed form solutions for the expected free energy depending on what those distributions look like.

So there's some work on doing this. For instance, Magnus Kudel has a paper in Entropy where they compute the expected free energy in linear dynamical systems, basically where all these A's and B's are represented by Gaussians.

So they call this like a linear controllable dynamical system.

And they also have to make some assumptions about how the actions affect the B matrix, which really isn't a matrix anymore.

It's more like a continuous Gaussian.

And they find that the normal terms you get with the expected free energy, like the information gain, those a lot of the interesting things that we get with when we use categorical distributions, those terms actually disappear when we use Gaussian.

So you actually don't get the nice like information gain seeking or information seeking terms that you would

normally get by using categoricals.

But there's other ways to get around that. Like if you use neural networks, you can still use things like sampling approaches to compute expected free energies, where you basically like sample a bunch of possible trajectories.

And then you can use the samples like kind of Monte Carlo style to compute expected free energies.

And a lot of the groups that have done like scaling active inference to deep neural networks, they've used that kind of approach.

So like Alec Chance's paper Scaling Active Inference does stuff like that. A lot of the stuff out of like Tim Verbalen's group.

I don't want to leave anyone out. I think like in NOR has done obviously a lot of, NOR Sagita has done a lot of active inference with deep neural networks.

As a FIRO's found to us, like a lot of these people were actually trying to apply active inference to deep neural nets.

They have to basically come up with ways to compute that expected free energy that are different than the way it's done in PI MDP.

Because in PI MDP, you can exactly compute that expected free energy, which ends up just being a bunch of matrix vector products and then a summation.

So it just becomes more difficult when you use more complex distributions, but it's by no means impossible.

You just have to, you have to come up with some kind of approximation.

But there are ways I think you can get around that, which involve what people have been calling hybrid models, where you might have deep neural networks that feed into a POMDP at like some higher layer.

And then in POMDP space, like a discrete categorical space, you can still compute expected free energies and free energies.

But you can still take advantage of at lower levels, like high dimensional neural networks to project your data into this low dimensional space.

So you do all the EFE active inference in this lower dimensional POMDP space where everything's exact.

But you can still take advantage of the nice dimensionality reduction and feature extraction properties of neural networks to first like pre-process the observations.

So that's something I'll talk about at the end, because we've made some progress on the first steps towards making that possible, which involves basically making all of POMDP auto differentiable so you can like train neural networks that are attached to POMDP models.

Yeah, it's a great question though.

Awesome. I see this as like the body plan of generative models.

And maybe what's a leg today is an antenna tomorrow or the leg gets longer or thicker.

There's a lot of different ways to swap out and compose different parts of the model.

So thanks. Carry on.

Cool. Yeah.

So yeah, now let's talk a little bit about the traditional software for active inference research, which I'm sure everyone on the call right now is familiar with.

The original way it was done was basically Carl Friston and a few others wrote a bunch of MATLAB scripts,

part of the SPM package,

which is originally developed for like neuroimaging data analysis and statistical testing.

And there's a sub package called DEM or dynamical expectation maximization, which is used for not only active inference, including continuous active inference, but also just like generalized filtering and fitting nonlinear state space models from empirical data.

So within that we have all the, this kind of discrete active inference toolbox.

Most of the functions of which are prefixed by SPM underscore MDP.

So the main function that basically does everything that PYMDP does for the most part is, is containing one function called SPM MDP VBX.

And kind of in a tongue in cheek way, we often joked around that PYMDP is just a package that implements that one function, because that one function essentially does active inference and learning and all the message passing just in, in one call, which in many ways is very nice because you just have to pass in a PYMDP to it.

And then it, in a kind of black box way, gives you all the belief updating and history of actions and everything that, that you need.

There are, despite its elegance and it's like usefulness and how robust it actually is, because you can pass in any agent with any generative model and it'll pretty much work.

The issue is because it's just a single function, it's not very modular.

So it's very hard to flexibly compose different subcomputations of an active inference process.

So if you want to do some bespoke active inference application, where like, oh, before I do hidden state inference, I want to update the parameters of the A matrix and I want to do it in this particular way.

It's very hard to do that in SPM, MDP, VBX.

So what often I see happening, and I did this myself in my master's research is I ended up having like five different versions of this function that have some little like specialist prefix for the specific thing that I was doing.

And then that just ends up being like kind of not efficient because you're creating a bunch of boilerplate code and then you have one function that does the particular little version of active inference that you want to explore in your project.

So just by modularizing this one function, even within MATLAB, you could do this.

You would basically make the thing a lot more flexible and save a lot of copying and pasting of code.

Another issue is that inference and policy selection of fixed.

So you can't kind of compare and contrast different message passing approaches and action selection routines.

So there's just one way it's done in SPM, MDP, VBX, which is called marginal message passing.

And so it's hard to like compare that.

Like what if you change the message passing algorithm?

Whereas in PIMBP, we have it right now to message passing algorithms that you can kind of side by side compare.

And then we hope to add more in the future, which is very easy because it's just a modular thing.

And then because of what we were talking about in the beginning with it, just by virtue of this being in

MATLAB, it's harder to synthesize with other frameworks like in reinforcement learning,

OpenAI gym and deep neural networks like in TensorFlow or PyTorch or JAX.

So just the fact that it's in MATLAB already limits it.

Of course, people have found ways to do like cross language, cross platform code, like moving from Julia to MATLAB to Python and back.

But it's just a lot, a lot more heavy lifting involved in that.

And then I'll just lay out some kind of pros and cons of MATLAB and Python.

So one of the things I do like about MATLAB is it's very easy to get started with array programming, has a nice editor.

There's a lot of use and history in computational neuroscience.

Like even when I gave this tutorial at a course on computational psychiatry this past year, I think the majority of the tutorials were still using MATLAB.

So there's still a good reason to use MATLAB just because of the history and the number of packages that are well suited for doing neuroscience and psychophysics stuff.

So there's that advantage, just kind of like almost momentum or legacy advantage.

But of course, there's issues like as proprietary.

I think that's the biggest one you need to pay one way or another to to use MATLAB.

And there's not as much community driven development.

I mean, there is some like the file exchange, but it's not the same level as something like Python.

And then Python kind of can compete with MATLAB because it has array programming in the form of NumPy.

Of course, it's open source.

It's been widespread adopted across tons of fields, not just academic ones, but lots of commercial application.

And there's a lot of community driven development in the form of packages like PyMEP.

But one disadvantage is for me, at least it wasn't as easy to get started with MATLAB as it was with MATLAB because you often have to like install a bunch of things and learn about virtual environments.

And you have to like learn about a lot of more programming stuff before getting started with using Python.

So I think that is one reason that MATLAB is actually still useful as a pedagogical tool like the time between getting MATLAB and actually doing programming is pretty short, which is nice.

So yeah, the package is called it's within this infer actively GitHub organization is called PyMEP.

And you can just pip install it like in a virtual environment or just in your base installation of Python.

And then once you have a PyMEP installed, you can kind of import and use in a flexible modular way, all the different sub packages or sub modules like the agent module, which basically just implements the agent class.

And then there's different modules like inference and control and learning that are independent of actually an active inference agent.

And you can just use them to do message passing for hidden state inference or to compute expected free energies on possible policies or to compute updates to parameters.

So all those things can be composed and flexibly kind of you can make kind of Frankenstein active inference agents by composing all these things in the bespoke way that you desire.

And a lot of it, the main workflow of PyMEP comes down to specifying the generative model in the forms of these discrete arrays and then plugging them into the brain of the agent and instantiating an agent.

So that is pretty much encapsulated by these two lines at the bottom, import the agent from PyMEP.

And then you just build one by plugging in the A, B, Cs, and Ds.

And then you have this agent object that you can use to do hidden state inference through methods like `infer states`, `infer policies`, which is the agent internally computes the expected free energy of its policies.

And then you finally can sample actions.

So these three lines are the kind of the main players of any active inference loop, state inference, policy inference, and then action selection.

And you just kind of tie those in a loop over time to instantiate an active inference process.

And the kind of the circularity of the action perception loop comes into play with the inputs to the agent, which are its observations, and the outputs of its action selection, which are its actions.

And of course, you need to use the last action to then get a new observation and that you do by plugging the action into some environment.

This is also in the active inference literature often referred to as the generative process.

So the actual world out there that generates your data.

So this is the kind of classic action perception loop you'll see in any active inference agent.

And this is also not even unique to active inference.

This is just how reinforcement learning problems are generically framed.

So like OpenAI-Igm uses a very similar control flow or kind of like, yeah, sensory motor loop.

So yeah, here's an example of just doing that, like import the agent, set up the generative model, and that's the hardest part.

So I kind of conveniently put ellipses after the dot dot dot, but that's actually where most of the code is going to happen is building the generative model.

Build the agent, you build some environment, which you can either import as one of the stored PINDP environments,

or you could get it from OpenAI-Gym, or you could just create your own environment.

So this would be your code that actually describes how the world works out there, the world that the agent is interacting with.

And then you can implement a timestep of active inference with those few lines of code right there.

And that would all be kind of those last lines, like eight through 15, those would kind of be wrapped within a loop over time.

Yeah, this is just more examples of like, this would be a quick way to, in this example, we're not even running active inference, but we're just using one of the message passing algorithms from the ALGOS submodule to do hidden state inference.

So in this case, I just created a random A matrix, I created a random observation, and I gave, and I made up a random prior, and then I can just do one like fictive update of what an agent might be doing during hidden state inference and optimize QS, or the beliefs about hidden states.

So this is an example of how you could actually use PyMDP to just do generic inference on hidden Markov models.

You don't even need to use it within an active inference loop.

You could just use it to do like statistical inference on a hidden Markov model with the particular algorithms that we've provided.

And then of course, one of the advantages I talked about over the SPM is that you can create customized active inference processes.

There's a lot of like extra arguments to the agent class where you can kind of turn on and off different parts of the reward function or the expected free energy for an agent.

So for instance, in this agent, the agent does not incorporate expected utility, which is the kind of reward C vector driven component of action selection, but the agent does use state info gain and parameter information gain, which are two other components of the expected free energy.

So there's a lot of ways that you can kind of create a bespoke active inference agent that doesn't, that will behave in different ways depending on these kind of keyword arguments that you provide.

Yeah, here are just more examples.

Like in this case, we're using active inference agent just to do hidden state inference and no actions at all.

So the agent is just inferring hidden states. It's updating its beliefs about the A matrix and it's updating its beliefs about the D vector or the initial hidden state.

And so you can, you know, just flexibly put together all these lines and create an agent that does whatever you want.

It doesn't even need to be acting in the world technically.

Okay. So now I'm going to show a few examples of PIMDP in action.

So this is one that's a classic active inference paper. It's one of my favorite papers. It was, I think it's called scene construction and active inference where it's describing a task where an agent has to, and this was used to actually model human data in a psychophysics task.

An agent has to gaze contingently uncover two out of four quadrants that have particular images in them.

And there's four quadrants. And there's four quadrants total and two of them have the images of interest in them.

And if the two images in this case are a bird and a cat image, then that's an example of a flea scene.

So the agent basically, or the human has to categorize the latent scene, which is simply defined by the combination of two images, and then categorize the scene.

So it's basically a categorization task, but the agent needs to gaze contingently uncover a sequence of cues before it knows what that category is or what that scene is.

So it combines the epistemic components that we all know and love about active inference, trying to uncover the hidden state of the world by actively sampling it.

So in this case, they're sampling the world by moving their eyes to different quadrants to uncover what's behind them.

And then actually choosing what the true category is based on what it learned about the world.

And that's where it's trying to maximize utility because there's some reward associated with categorizing correctly.

So this is an example of that which was originally done in MATLAB and I just re-implemented in PyMVP.

And this is another example where now the scene is the feed scene.

So the cues are in the lower two quadrants in this example.

So the agent has to look around the different quadrants.

It finally sees that there's, okay, there's a bird in the lower right, seeds in the lower left.

So this must be the feed scene.

And just to show an example of what does that actually look like in active inference?

Like what does the code look like for that?

So the first thing you would do for this one is you would set up your agent, which again, you just throw in the A, B, Cs, and Ds.

In this case, I used a particular message passing algorithm called marginal message passing, which is the same one used in MATLAB.

You set up a policy depth and an inference horizon, which is kind of like a memory, like how much of the past observations you take into account.

And then you set up a environment.

This is like the external world that the agent will be interacting with.

In this case, I'm calling it the scene construction environment, which just tells me once the agent moves its eyes to a certain place, how does that action then determine what the agent sees next, which will be, you know, whatever is behind the quadrant that it decided to look at.

So those are the main two things. Those are the two sides of the action perception loop, the agent and the environment.

And then what it's often common to do is to get an initial observation by resetting the environment, which is a convention borrowed from OpenAI Gym.

You basically do `environment.reset`. It's like a method of the environment that spits out the initial observations.

And then it's often useful in these things to create like lists or dictionaries that have a mapping between the observation indices, which are like integers between zero and however many observations there are.

And then what those actually correspond to semantically.

So this is just a very common way to like, because all the POMDP and the environment will be spitting out or like ones and twos and zeros and all these like discrete indices, but it's useful to have these lists that you can use to kind of semantically map particular indices to things that are meaningful, like seeing the bird image or choosing the category one versus category two.

And then once you've done that, you just write a loop over time where you're basically performing active inference, which consists in hidden state estimation and policy inference.

So you sample an action, which then gets fed back into the environment to produce another observation.

And then that happens over time.

So that's the whole action perception.

So this is, yeah, so that's, it's deceptively simple how like short it looks, but I'm kind of glossing over something that I'll talk about later, which is, I mentioned earlier, which is as easy as this looks, the hardest part is actually done way before any of this happens, which is writing down the A, B, Cs, and Ds.

That's by far the most time intensive and like complex part of active inference is actually writing down the generative model.

Once you have the generative model written down, then the rest basically is like clockwork.

You just have to link the agent to the environment and then just run like five or six lines just to actually implement the thing.

But the hardest part is writing down those A, B, Cs, and Ds in the beginning.

I'll show another example, which I kind of call T-Maze on steroids.

So in the classic T-Maze task that I forgot what the original paper was, but it's something that's been very popular in the active inference literature for a while.

So in the last one, I think Carl came up with maybe in 2015 or earlier even, is you have an agent, a mouse that has to visit two potential sources of either reward or punishment in its environment.

And it doesn't know which arm of this T-Maze contains the reward.

So it has to visit a queue first before it knows which arm has the reward and which one has either no reward or like a shock, like a negative stimulus.

In this one, I just kind of spatially extended the T-Maze so the agent has to now visit a sequence of queues, each of which reveals the location of the next queue, in order to figure out the final queue, which is just the location of the cheese versus the shock.

So this is another example where the agent first goes to queue one, then it knows where queue two is, and then it knows where the cheese is.

So I call this epistemic chaining because the agent doesn't actually have to plan its route all the way to the final location of the cheese.

All it has to do is get to the next queue, which then reveals where the next queue is, which finally reveals where the hidden location of the reward is.

So you're kind of using epistemic value or curiosity to allow an otherwise temporally shallow animal to plan its way to a distal reward or something that it can't plan to get to a priori.

And again, this is just an example of what that would look like in PyMDP, where it basically looks exactly the same.

It's just that the environment and the generative model are different, but the general flow of the code is always has this kind of classic pipeline.

Okay, so now I'll get to the kind of most important part.

And I think the biggest source of confusion with active inference is that the hardest part is the generative model.

It's all the complexity comes into encoding the agent's beliefs about the world.

So how do I write down the A, Bs, and Cs, and Ds?

In paradigms like deep neural networks or like unsupervised learning, you don't have to write down the model.

The neural network learns the model by just observing loads and loads of data.

So it's less sample efficient, but you don't have to encode as much to begin with.

So this is kind of the kind of coincides with a larger divide between model-free and model-based approaches.

With deep neural networks, you're effectively issuing the sample, the statistical complexity of having to write down the model by just sticking together a bunch of nonlinear function approximators,

and then just learning the beliefs that the agent has about the world by just bombarding it with data.

The same thing goes for deep reinforcement learning, like deep Q learning.

In active inference, the agents are much more sample efficient, in the sense that they don't need to train on like billions of data vectors.

But on the other hand, there's more investment on your end as the modeler, because you have to write down explicitly what the agent's beliefs about the world are.

You don't just equip it with something as generic as a convolutional layer, and some values and stuff, and then let it learn.

You actually have to hand code that.

I think this is one of the biggest differences between model-based reinforcement learning, where you actually encode a Bayesian generative model of the world, and kind of more model-free or data-driven approaches.

But there is, it's not such a dichotomy.

There are ways to kind of combine the two.

So just to kind of show that very specifically, for instance, in this scene construction demo that I showed like a few slides ago, if you just look in terms of sheer lines of code, which one took more code, you can kind of use the amount of lines of code as a proxy for statistical complexity, or how much information is contained.

So the simulation itself running the active inference loop was like 15 lines of code, just like, and that code itself is already very generic and not specific to the scene construction demo.

Writing the generative model itself, that's where all the heavy lifting is done.

That's where all the information that's specific to that task gets encoded.

So for instance, I just look at how I created the A matrix, the beliefs about the observation mapping for the scene construction demo, that already is like way more code than just running the entire active inference simulation.

So just like by the sheer amount of code, you can already tell, oh yeah, there's a lot of assumptions and information that's being baked into the generative model. And that's where most of the heavy lifting of active inference actually comes from.

Yeah, so I just think it's important to remark on that,

because that's like a really key thing that I think anyone who wants to start working with active inference models

and in discrete state spaces should kind of wrap their head around is that the model does most of the work for you.

The expected free energy, yes, is a very interesting objective function that has many advantages, but most of the power of active inference comes into writing down what your agent's beliefs about the world are.

And then once you have that, then all of the rest kind of just does the work for you, because the PyMDB code is very generic.

What's not generic is how you encode the beliefs about the world.

Okay, so now I'm kind of finishing up.

Maybe we should dwell just on that bit for a second.

Does anyone have thoughts or comments or questions about this?

If not, I can just proceed and finish off.

Daphne or Jakob or I'll ask one.

I don't have any questions.

All right, well, you've emphasized the specification of the agent generative model.

And how about the other side of the coin?

How do we specify a generative process?

How do we specify the environment for the agent?

That's a really good question.

Yeah, basically everything I said about the generative model kind of applies to the generative process as well,

except the agent's interesting behavior.

Yeah, I mean, you could think of the generative process as driving a lot of that too.

I guess the bottleneck is the generative model,

because if you create a really complex generative process,

so a really complex environment that has all kinds of fancy nonlinear dynamics,

but the agent's model of the world is super, super simple.

So it just believes that there's a light switch that's either on or off,

then the possible behavior you can get from such a simple agent is limited by the complexity of its generative model.

So a very simple generative model will still not show very interesting behavior,

even if it's embedded in a complex generative process.

But the most rich dynamics will obviously happen when you have both a complex generative process and a complex generative model.

So all the work in building the generative model, which I would say is the first line up here,

can also be matched by a lot of work in generating the generative process as well,

which in this case is this epistemic grid world environment,

which is just a set of rules that says,

when the agent is in the queue location, show them the queue identity.

Like this one is relatively simple.

But one interesting thing to think about,

and I'm sure like you, Daniel, have thought about this when it comes to,

you know, your work on active inference and collective behavior is,

the interesting thing about multi-agent behavior is in that case,

the generative process are the actions of other agents.

So the generative process, my generative process are actually the outputs of another active inference agent. So that's one of the most complicated things and what Daphne and I had to grapple with when we were doing the, and Mao as well.

Mao was the first author of the epistemic communities work, this like social network echo chamber stuff.

In that context, the generative process is a little bit more difficult because the process itself is consisting of other active inference agents that are also acting.

So the control flow of that code will look a little bit different where you're going to have to loop over all agents, get actions from them,

and then use those actions to parameterize the observations for all the other agents.

I mean, that's just a generic statement about multi-agent simulations in general,

but it's particularly interesting when you think about agents trying to model other agents,

because almost necessarily every active inference agent will have an impoverished model of how the world works,

when the way the world works is a bunch of interacting active inference agents.

So you're going to have to kind of necessarily equip each agent with a more simplified generative model,

unless you want them to all have like infinite recursion depth and be able to simulate in their own generative model,

the generative models of every other agent.

So yeah, that was kind of a tangent about the multi-agent case, but I think it's just an interesting complex to think about the tension between the generative model complexity

and the generative process complexity and how they kind of mutually constrain the behavior of each other.

Okay, shall I just proceed? So yeah, the last two slides I think is the kind of exciting stuff.

So here are a list of things that we'd like to do with PyMDP in the future.

I'll just go through them and then I'll dwell on a few that I think are the most important.

So one is fitting PyMDP models to empirical data.

So I've interacted a lot with people from the computational psychiatry community

who are interested in actually creating models of behavior, often human behavior, that are their POMDP active inference models.

And one of the biggest, I think, limitations of PyMDP right now is that people can't use PyMDP to infer the active inference parameters of like a human subject that's performing some task.

That's what you can do in SPM right now, but unfortunately you can't do that in PyMDP.

So this is like really high in the priorities list. I think this is what will help PyMDP actually become competitive to SPM for the communities that are interested in fitting PyMDP models to data.

So these are like kind of more empirical scientific disciplines like computational psychiatry.

Other things is I think we need better interfaces for actually generating and constructing generative models.

Right now, all that code involved in building A and B matrices, that really becomes the bottleneck for anyone trying to do active inference.

And in large part because constructing those arrays for complex generative models can be a real headache. You have to do all this weird multi-dimensional indexing because like if you have like a bunch of different interacting variables in the world, you have to create massive multi-dimensional arrays that have different numbers of extra dimensions that correspond to all these possible contingencies in the world.

It kind of becomes a massive lookup table that you have to encode all the relationships between variables.

And so I think there might be, this is an ambitious project, but there might be ways to actually create kind of UIs like user interfaces that help people build generative models by like asking them a sequence of questions.

For instance, do you want this variable to affect that variable?

And then depending on their answer, you can kind of prove parameterized part of the A matrix or something.

And then the actual structure of the A matrix gets windowed down through a sequence of kind of yes, no questions about the different contingencies in the world.

Another thing is interfacing with OpenAI Gym, which we kind of already have done.

Like there's a few examples where we've done this.

I haven't put these on the, on the Infraactively GitHub yet, but this is something that's an open, this is like a very obvious and easy thing to do because like we wrote our environment class as if it was based as if it was a gym environment anyway.

So once you do that, it will open up to comparing active inference agents to all kinds of reinforcement learning algorithms.

So hierarchical models is a big one.

So basically allowing you to stack hierarchically active inference agents within each other.

So like, yeah, there's a lot of temporal depth that you can get out by stacking active inference agents into hierarchical things.

So like one timescale of inference and planning is happening at a slower one, slower than a sub faster timescale.

We need more demos that demonstrate parameter learning.

So you can do updating of A, B and D arrays.

I don't think you can update C so far.

And I know this is something Daniel, you mentioned to me, which is people in your, in the Active Inference Institute are generally interested in updating the beliefs about the generative model parameters basically.

And then there's things like sophisticated inference, which is a kind of a more recent version of planning under active inference.

That's kind of interesting and has some computational benefits to it.

And then hand in hand with sophisticated inference goes this thing that people have developed, have had to deal with in deep reinforcement learning for a while, which is how do you tame combinatorially explosive policy spaces?

So when you're doing deep planning over time, the number of policies is exponential in the number of time steps that you plan in.

So that's, there's various techniques for dealing with that, like Monte Carlo tree search, which I think some people like Théophile, Champion, and others have already tried to start to implement in their own

implementations of POMDPs.

And so along these lines, I want to just point out that we're actually very close to getting this working now.

So fitting POMDP models empirical data.

So there's a branch that Dimitri Markovic and I have been working on called the Agent Jax branch.

We've basically written a backend for POMDP and Jax, which not only lets us use a bunch of statistical probabilistic inference techniques from like NumPyro to invert or infer the parameters of POMDP agents from like data, for instance, collected from human participants.

But the fact that it's also backended in Jax means that POMDP is now fully auto-differentiable.

So it means you could stack a deep neural network layer onto the, like before the A matrix layer of a POMDP agent.

And then you could use something like the variational free energy or any other objective functions to automatically train the parameters of a neural network that's linked to a POMDP agent.

So this, I think, just re-implementing it in backends like Pytorch and Jax is like a huge benefit because this will really allow you to extend POMDP to much more high dimensional state spaces by linking up deep neural networks to various components of the agent's body as you were describing it, Daniel.

Daniel. And so we originally did this just to allow you to do fitting of empirical data, but it comes with a side benefit of allowing you to differentiate and pass, like back propagate gradients that you would use for updating deep learning models, which is, I think, really exciting.

So that's like almost done. I mean, yeah, like we're very close to putting up a notebook that actually does that. If you look in the agent Jax branch now, it's not very organized, but that stuff is now there and implemented.

Daniel. Another thing is Noor Sajid and I have actually implemented some of the environments from her paper, Active Inference Demystified Compared, and we've actually done that with Pymdp in OpenAI Gym, like in the frozen lake environment is a popular one for simulating B matrix learning.

Daniel. So that's something that's also just like, we've done that and we need to like upload that or I don't know, write a short paper or do something with that because so there's a lot of like these different tendrils that have been explored.

Daniel. It's just a matter of pushing forward and actually putting them up on the Pymdp repo.

Daniel. Yeah. And then these other things I would like to find time to do, but I just haven't. But I mean, as I kind of said in the beginning, this was a very much a collaborative effort. So I don't also want to be necessarily the one who's like doing all of this because I think it also is healthier for the development of the package if different people are kind of taking the lead on different things and developing it in their own way.

Daniel. So that's something I also just generally like to encourage is for all kinds of interested people to get involved in the development.

Daniel. And I don't think Brennan is here. But Brennan Klein also who's a postdoc and research scientist at Northeastern University at the Network Science Institute. He started these Pymdp fellowships.

Daniel. So you got funding from Northeastern, I think also the Templeton Foundation to fund people to work on Pymdp development or Pymdp adjacent projects.

Daniel. And I think the first round of applications is over, but it would just be, this is a good opportunity to advertise that. I think there's going to be another cohort in the summer.

Daniel. So this is a kind of seemingly ongoing source of funding. So it's just nice to see that other people are

kind of trying to push Pymdp in their own directions.

Daniel. So that's just an encouraging development that I want to keep everyone apprised of.

Daniel. Oh yeah. And then I'll just end by

Daniel.

Daniel.

Daniel.

Daniel.

Daniel.

Daniel.

Daniel.

Daniel.

We have another, a new demo that's not listed here, which is about just calculating the variational free energy in discrete categorical models, which is based on a demo from Ryan Smith and Christopher White and Carl Friston's paper on that big tutorial paper on active inference.

So I re-implemented one of the demos from that paper, and now that's also in the docs.

Yeah, so you can open all those demo notebooks in Colab and just step through them.

You don't need to have even Python installed on your computer to use them.

You can just open the links in Colab and step through the code and build your own active inference agents.

So it's just useful for pedagogy.

That's why I mentioned if you're just getting started, I would definitely recommend going to the documentation.

So yeah, thank you all for listening and for giving me the chance to talk.

It was nice to be here as always.

And I guess like, yeah, I listed at the bottom for the next live stream, we can go through

some of the demo notebooks, but we also could go through them now if there's time.

And we'll do discussion first and then just see if there's time.

Awesome.

Thanks.

Yeah, let's address some questions from Daphne and Jakob.

I'll ask some from the live chat.

And then perhaps you could share one or a few of the examples on the Read the Docs and we could just look structurally at what the anatomy and physiology is of a notebook.

So first Daphne or Jakob, any thoughts or questions?

Yeah, Jakob, go for it.

I'm wondering on the JAGS implementation, are there any requirements on defining the generative process at all?

Or is it just about defining the structure of the generative model that we then fit to experimental data?

And I guess this also relates to another question I had in scaling these models to state spaces or generative processes that we as modelers don't have the liberty to actually define ourselves, but we want to deploy and train these agents in generative processes that are already out there like in an online setting where you get categorical or discrete data coming in.

Yeah, totally.

So on the side of...

That's a great question.

On the side of...

Say I had a PineBP agent that had a bunch of deep neural networks attached to it.

And I wanted to train it in a deployed setting.

So it's like out there, you know, let's say it's an agent that's trading on the stock market or something.

It's like placing bets to buy cryptocurrency, let's say.

So in that case, in the same way to train a deep neural network on that kind of data, you don't need to pass gradients through the generative process, which of course you don't have access

You're trading on the stock market.

So in that sense, no, there's no requirements on passing gradients or writing up a generative process that is also auto-differentiable.

There is one case when you would want that, which is often in the case of empirically fitting PIMDP models to data.

Often one thing that you want to do is you have a bunch of like...

You basically have a history of actions and observations of a human participant.

You fit the model, the parameters of the PineBP agent that best explain the observed actions of your

participant.

And you know the observations because you're an experimenter who decided this person is going to see the sequence of observations.

So you can do all that without having a differentiable generative process or environment.

But then there's something in Bayesian inference that's called like a posterior predictive check, where you say, okay, given my inference about the parameters of the PineBP agent, then I'd like to roll out the expected behavior of this agent, given my best guess for what this agent's parameters are.

So that's called like a posterior predictive density, where you say, given my posterior estimate about the agent's parameters, what would it look like in the future under these posterior parameters?

And to do that in using numpyro, which is the probabilistic inference framework that uses JAX as a backend, you would want to have a generative process that is also auto-differentiable.

But in that case, I expect that writing those generative processes would be easy, because that would be in the case of fitting a human's behavior to experimental data where they're like in a controlled task environment.

So in a case of trying to fit someone's parameters, like the value of their C vector, and they're performing that scene construction task where they're circadian around, you could write the generative process because you as the experimenter developed the psychophysical task that they're interacting with.

And then you could write that also in JAX when you're doing the modeling, so that when you're doing these posterior predictive checks, you know that that's also written in JAX and that you can compute those quantities.

But in a deployed setting, you're not going to even be able to do any kind of posterior predictive check in the future, because you don't know how the environment actually works, right?

So you'd have to, that wouldn't even be something that you tried to do in the first place.

But yeah, so there's nothing inherently stopping you from just, as long as the models are differentiable in the same way they are with deep neural networks, there's nothing stopping you from just throwing them into an environment where you don't know how the rules of the world work.

Yeah.

Yeah.

The inexorable logic of natural selection or free energy minimization or just non-equilibrium systems, whether or not they know what's out there, either it's going to work or it won't.

And if it fails, it fails.

And the computational environment allows us to exist in this kind of gray zone where the computational agent might be quite poorly adapted to a given deployed setting, but the computer program will still run.

But of course, we're interested in cases where the computer program runs and the agent is able to event some kind of meaningful or even useful behavior.

Exactly.

Like we could all imagine very simple PIMDP models that would do terribly in some tasks, right?

Just like a dumb model that has two hidden states that it believes just stochastically switch between each other.

And then you give it the task of making investments in like a 10 stock portfolio.

And of course, its model is not fit.

But the promise of applying big deep function approximators to different ends of the PIMDP agent means that hopefully you could then learn a good generative model and then still combine that with some lower dimensional generative model up top that can do all the nice inference and planning with active inference.

But it can deal with high dimensional or ugly, hard to tame observation and action spaces by using deep neural networks.

So I think that's really the way to just in the same way deep learning has gotten that to work in a lot of cases.

This is the way to kind of do it with PIMDP models as well.

Daphne, any remarks or I'll ask a few from chat.

I don't really have any questions, but I do think that it is really fascinating.

And like, I think it's really exciting to think about.

Yeah.

Like, as you said, like learning the generative model of like an agent learning its own generative model given like some real world data to like figure out like what is them.

And I guess like in terms of what you were talking about with like the function approximations and NumPyro and stuff, is that still like what?

Like are you guys still working like from that code base and then finally convert those or how we kind of started from scratch?

Yeah, we pretty much started from scratch.

The PIDCM thing.

I'm actually not sure what the like the IP status of that is because we worked on it as part of nested minds.

So I don't have access to that code anymore, but that was more implementing like variational Laplace in which you were we worked on that together.

With variational Laplace in JAX, which is the way that you do gradient descent on free energy when you're trying to do inference.

What we're doing now instead of that is we're saying, can you rewrite a PIMDP model such that you can pass gradients through it, JAX, like accelerated gradients, and then use NumPyro to do all kinds of fitting routines, not just variational Laplace.

But you could use MCMC, you could use like NumPyro just has a massive library of different probabilistic approximate Bayesian inference techniques.

So you can kind of throw the kitchen sink of NumPyro inference techniques at a PIMDP model.

So the challenge there is just rewriting a PIMDP model.

So you can define like a likelihood function that goes from like the PIMDP parameters to the observations, which in this case would be the actions of the agent.

And in order to do that in a way so that can like play friendly with JAX, we just had to make sure that all the interior functions of the PIMDP agent, like the inference, the planning, action selection, all of that was written in JAX, so that you can pass gradients through it when computing like likelihood gradients effectively.

So yeah, that makes a lot of sense.

Yeah.

All right, I'll ask a few questions from the live chat.

So first, most descriptions of active inference across the literature are written in terms of matrices, but PIMDP clearly works with tensors.

Do you have a good reference for how the operations are different when generalizing the equations for matrices to tensors?

Hmm, that's a really good point.

This is one of the things that actually frustrated me a lot when I was first learning about active inference was I noticed exactly what this person asked is that a lot of the basic operations are written as if there's only single dimensional hidden states and single dimensional observations.

So everything is like they said matrix vector products and matrix math, but what we're really doing is tensor multiplications and tensor products.

So in terms of references for how that works, there's, yeah, so essentially there's nothing super qualitatively different.

These tensor operations that we do in PIMDP are basically just fancier ways of expressing sums of matrix multiplication.

So there's the mathematics of it is all still standard linear algebra.

It's just the way we represent these high dimensional matrices as tensors is just a more efficient representation.

So mathematically, it's nothing too crazy.

The way I learned about how that worked was just by staring at functions in MATLAB for like a year until I just figured it out, but it wasn't easy and there's definitely better options out there now.

So one reference off the bat I'll recommend is the appendices of PIMDP, the archive paper.

So that, like I think appendix A or all of those appendices basically deal with the full tensor factorized version where it's not dealing with matrices, but we're actually indexing into higher dimensions.

Another one, which I think originally discusses the tensor products and the tensor factorization is in an appendix of, I think, active inference, curiosity and insight, which is where they first talked about like novelty and parameter information gain.

That's a paper. I'm sorry, I don't remember the year it came out to either 2017 or 2018, but I know the paper title is called active inference, curiosity and insight.

And in one of the appendices, they actually do the full tensor based mathematics.

And then finally, another good reference is a recent paper that was, I think, headed by Théophile Champignon.

I'm just gonna try to find it real quick because it's, I don't want to forget this.

Maybe I'll stop sharing my screen.

Did I stop? Okay.

It stopped.

It's a really good reference that has appendices about doing tensor math for active inference in particular.

We also recently learned about the branching time active inference, which speaks to some of those questions of computational complexity and all.

Right. Yeah, I should have mentioned that.

That's like probably the most promising approach to date about the, to finessing the computational complexity

of active inference.

So yeah, this, this one is by Théophile Champignon, Marek Gresh, I guess that's one of his advisors and Howard Bowman, his other advisor.

And that is called multimodal and multifactor branching time active inference, which I just posted.

So I haven't read this myself, but I've heard from other people.

Like I think Alec Chance told me that the appendices are really good for the, the full tensor generalization of active inference.

All right. Awesome. Well, I've added all of those citations mentioned into the YouTube live chat.

Awesome. Thank you.

I'm going to ask a following question from Fausto.

Using a Jack's backend makes it easy to wrap PYMC3, PIMC3 around it.

E.g. to have PyMDP as an operator to use in a PIMC3 model.

Is there any plan to do this?

I ask because there's a growing Bayesian community in Python around PYMC3.

Hmm. That's really interesting.

I didn't know actually that PYMC3 was also had a Jack's backend.

So I'm, I don't know, to be, to be honest, I'll start by saying that.

I don't know because my introduction to like probabilistic modeling in Python was through Dimi, Dmitry Markovic, who basically sold me on NumPyro.

NumPyro is the way of the future.

And I know NumPyro has a Jack's backend.

And I think NumPyro and PYMC3 occupy a similar place in that ecosystem of probabilistic inference in Python.

I don't know how models are specified in PYMC3.

I am assuming it's not too dissimilar from how it looks in NumPyro.

And because all the low level backend is now written in Jack's, I can't promise this, but I would assume that you could just write a PYMC3 model in the same way we wrote a NumPyro model that wraps PYMDP functions, but only the PYMDP implementation in Jack's.

So if PYMDP only depends on Jack's at the low level, then yes, it certainly can work.

But I don't know, is there anyone else here who has experience using PYMDP and might know?

Because I just don't know enough about it.

I use PYMDP a little bit, but I think it is like, yeah, as you said, like quite similar to NumPyro.

I think that probably you'd be able to do the same things that you're doing with NumPyro, like integrating NumPyro with PYMC3 as well.

Ah, cool.

And so it's primary backend is Jack's?

Is that?

I didn't actually know that.

Yeah, I didn't know that either.

Yeah.

In comparison with the matrix multiplication of MATLAB, what makes you excited about the probabilistic programming direction and all of these packages and approaches that we're naming?

How does the probabilistic programming differ from just writing out the matrices and calculating them on paper?

And why does that have some promise for implementing active inference models?

Hmm.

I think the biggest advantage of the probabilistic programming is not necessarily for simulating active inference agents.

For simulating active inference agents, for sure, the matrix multiplications are sufficient.

Having it in Jack's makes it much more scalable.

So you can use all the vectorized operations to run like tens of thousands of active inference agents simultaneously because you have these highly optimized just-in-time compiled functions in Jack's that allow you to...

It just basically speeds things up in order of magnitude.

But the probabilistic programming angle is not as much for simulating active inference processes as it is for doing inference or fitting models of active inference agents to empirical data.

So say I observe an animal or a person doing something.

Now, what we can do, which we can only do in SPM, but we can't do in PiMVP yet until now, is we can take a sequence of someone's behavior and then infer the best parameters of an active inference model that explain their behavior.

So given how someone decided, I can say, oh, their A matrix must look like this or their C vector must have this precision to it.

Like you could infer someone's risk sensitivity or their risk aversion given their behavior.

And the nice thing about being in probabilistic programming languages is there's so many different methods that haven't been really well explored for fitting active inference models in the MATLAB literature because almost everyone there uses this variational Bayes approach where you basically minimize free energy.

You use a Gaussian approximation to the posterior.

So you use a Gaussian type of variational inference.

You use a Gaussian type of variational inference.

Now that it's in NumPyro or perhaps PiMC3.

Again, this will be very soon.

It's not fully implemented yet.

We'll be able to throw all different kinds of probabilistic inference techniques that have their different advantages and disadvantages.

Like a big thing in probabilistic inferences is MCMC, Markov chain Monte Carlo inference.

It's supposed to give less biased posterior distributions.

Like there's advantages to using MCMC over variational approaches to approximate Bayes and inference.

And one thing that I haven't seen done, which I'd love to see, and a lot of people in the computational psychiatry community have complained and told me what they'd like to see, is like a side-by-side comparison

of variational Bayes to infer

PyMDP parameters or POMDP parameters versus like MCMC approaches.

So once you have everything in a probabilistic program framework, you can do side-by-side comparisons between all the different inference techniques that you wouldn't necessarily have if you were limited by a language where only one or two or three inference techniques are implemented.

So it's basically just taking advantage of all the work that people who've worked on NumPyro have done in implementing all these different kinds of inference methods.

Yeah.

Awesome.

And Fausta followed up the primary backend for PyMC is a Sarah, but the new version can use Jack's.

I might get around to writing the PyMC3 wrapper.

Once the Jack's version of PyMDP is stable, it sounds very doable.

Well, that's awesome.

If you see it as likely and you have the affordance, then just minimize your free energy and you won't be surprised when you do that.

Absolutely.

That's great.

That's promising.

Nice.

Jacob or Daphne, or I can ask another question.

Something for me right now.

Well, you mentioned message passing several times in the context of PyMDP.

So what is message passing and how was it used in PyMDP?

Hmm.

It's a good question.

So message passing in general describes like a set of algorithms that you can use to do an exact or approximate Bayesian inference.

So oftentimes in the, to make it very concrete in the, in the context of doing Bayesian inference about hidden states.

So what an active inference agent will have to do when they're faced with some observation, they'll have to combine messages.

Like one message corresponds to the message, the sensory information.

And then another message corresponds to their prior beliefs about the world.

And they use some algorithm to combine those messages to optimize a belief about the current hidden state of the world.

So in PyMDP, we have a very naive kind of computationally efficient doing way of doing that, which we just call naive or vanilla fixed point iteration, which is like the most simple message passing scheme you can think of, which is just, I'm actively filtering hidden states using my priors from the past.

So I say, given where I was at the last time step, where should I be now, given my B matrix?

And then I just essentially combine that with my incoming sensory message, which is just the observation passed through the likelihood matrix, the A matrix.

And then I just combine those two messages together.

And that resulting thing is my posterior distribution, my posterior or best belief about hidden states.

That's like the simplest form of message passing that has this very temporally shallow current evidence combined with prior to form the new belief.

It has this very kind of Bayesian flavor to it, right?

Where the best posterior is just a product of the likelihood and the prior.

There's also more advanced message passing techniques that are used when your beliefs themselves are more complicated.

So in the full implementation of active inference that's in the MATLAB version, agents don't just have a belief about what the current hidden state is.

They have a full predictive and post-predictive kind of tensor or cube of beliefs about all the hidden states in the future and all the hidden states in the past, further condition on all the policies I will potentially take or could have taken in the past.

So you have this massive like belief tensor stretching into future and past hidden states and further factorized by policy.

And when you have that kind of belief that you need to update, you have to use more sophisticated message passing techniques, one of which is called marginal message passing.

There's something called variational message passing.

And all these different message passing techniques are just essentially consistent in passing messages forwards and backwards in time, as well as across different variables that characterize the hidden state, which we call hidden state factors.

And the message passing algorithms are basically still amount to combining sensory information with prior beliefs, but they just have kind of more complicated trajectories through the space of beliefs in the future and the past.

There's people who can explain this much better than me.

I've implemented some of these in PyMDP, but I would refer people to, there's a really nice paper.

I think you may have retweeted it the other day, Jakub.

It's about, it's called mean field.

Oh yeah.

The paper comparing the mean field and Bethy approximation.

Yeah.

And the Bethy approximation.

Neuronal message passing using mean field, Bethy and marginal approximation.

Par Markovich, Keeble and Friston 2019.

It's a paper that a few of us have been walking through, looking at how the different free energy functionals look different under different approximations.

And it'll probably be a focal paper in 2023 for us to really dive into because a lot of these, um, vintage, let's say 2011 to 2019 papers.

Now packages and development directions such as PyMDP are facilitating these methods to be actually used.

And there's a huge wealth of conceptual possibility, proposed heuristics, exciting use cases, relevant other kinds of connections.

And, um, as you brought up earlier, it was very MATLAB bound to bring those kinds of connections into the last mile.

And then especially, um, the more granular or modular developments were under the umbrella of the SPM VBX, which prevented them from being meaningfully shared in a true distributed open science or decentralized science way.

And so that's why, of course, we've been so excited to work with and build on PyMDP and learn about it more because this is exactly the kind of composability of active inference agents and their different implementations that is going to be able to be worked on in a massively distributed way.

Somebody might specify a really interesting A matrix.

Somebody else might specify an interesting B.

Somebody else is going to link those together into a new kind of agent.

Someone else can implement it differently.

Somebody else can implement it differently.

Somebody else can implement it differently.

And so it brings like a natural kind of factorizes the process of developing these algorithms, which previously were almost always either fully MATLAB or, and or bespoke and very custom and fit for a given paper.

But not necessarily adaptable along the relevant axes that one would want to use for a modern, especially Pythonic setting.

So I think that's a great way to think about it.

I couldn't agree more.

I mean, that's a great way of thinking about it too, is like, by making modular flexible code that exists in the ecosystem of other packages, you're essentially factorizing the collective mind's representation of the task at hand, where then different parts of that representation can be worked on without having to pass messages or take into account what's going on across the entire network of distributed workers.

So like someone can write their own, you know, even better message passing algorithm and then just slot it in to use with PyMDP without having had to learn about how every little facet of PyMDP works, you know.

So yeah, that's a really important thing about just open science and modular software development.

So, nice.

Nice.

Well, in our closing minutes, of course, Jakob or Daphne, any remarks or questions?

And also any appetizer, what kind of models are people excited about?

And or what might we see in the following live stream, which will be in January 2023, model stream 7.2?

I would just say that I think that the notebooks are really, really useful.

So like, it's a really great resource for people who are trying to build a model and understand like what's going on under the hood of PyMDP.

And I think that it would be really cool to have an extension to those notebooks that also talks about learning the Dirichlet parameters for the A and B matrices.

I think that that would be really, really cool.

And thank you so much, Connor.

You wanted to talk.

Yeah, thank you for coming.

That's a really good point.

And this is something that Daniel also said earlier in the emails that updating A and B is like, it's very, very under documented right now.

And I think that would be because that is a form of learning the generative model that right now we don't, it's not the most sophisticated way.

You still have a fixed number of rows and columns.

So you make some assumptions, but that is like a flexible way when the agents are themselves learning the B and A matrices.

So yeah, we should definitely, maybe that can actually be a, I can just add that into the notebook that I was planning on showing this like a kind of epistemic two armed bandit task.

We can just add in some A or B learning to that and just show how that works or make a new notebook that uses that.

Cool.

Cool.

Eventually for the textbook and for every paper, it would be amazing to be able to see the code, the analytical representation, a graphical representation and different natural language representations, because they're all formally connected and they could all be rendered as such.

And that would really, one might expect, increase the accessibility and rigor of a model and help us compose and connect across different domains and just welcome and recognize many different kinds of learning and modeling.

Absolutely.

Jakob, any thoughts or questions?

No questions at this point, but also just thanks a lot for the awesome presentation.

And I'm really excited about all the emerging integrations and use cases that will undoubtedly spring up.

We've been, we started exploring NumPyro as well and kind of discussing how that can be used for scalable active inference models.

And I'm really excited how PymDP will inter, interoperate with all of these different integrations and it will be very exciting to, you know, exciting to use.

Thank you.

Connor, any penultimate words?

I guess maybe just in the spirit of what you were saying, I can just show a skeleton of what we could go through next time.

Oh, great. All right. We see it.

Yeah. Is that, is that okay?

You can also see.

Looks great.

Looks great.

So yeah, basically this is a colab notebook.

So I encourage anyone to go to the PymDP tutorial website and each of the notebooks, like Taffney was saying, they have, they're pretty useful and they have colab links associated with them.

And you can just open the link and then explore this one, I think is part of the agent API.

Yeah, this is the same one except I showed this one recently at the course on computational psychiatry.

So I've updated a little bit so I can share this with you, Daniel, and then you could either put on the discord or wherever.

This one's a little bit more updated, but the basic one here will still.

That you can access here will still show the same thing.

But in any case, the thing is you just opened up colab, you need a Google account.

That's the one limitation to use these.

You can install locally, infractively, dash Pymdp, import numpy, mat.lib.

There's just some imports here.

And then this whole, the spirit of this notebook is essentially just going through all the steps in setting up a generative model.

So creating your hidden state factors.

That's something we didn't really talk about is factorized representations in the context of Pymdp.

building the B array, which not only has, like, you can just do it yourself, but there's also these hidden cells with solutions to each of these things.

And then the main brunt of this notebook before running active inference is just stepping through and actually initializing the entries of the A, B, C, and D arrays.

And next time I have some slides to go along with this so we can basically go between the slides and the actual code.

Same thing with the A array.

So you, like, have some representation of what you want the A matrix to look like.

And then you go into the code and actually build it out.

And you do that sequentially for each of the components of the generative model.

And you're plotting them along the way so you can see what it looks like after you've built it.

And then we actually implement, after you've built the generative model, then you actually plug it in to an active inference agent.

So the first stuff is building a factorized A and B for a generic generative model that I think is just a more sophisticated grid world.

But then we actually, that's kind of the introductory task.

And then we go in and we actually build the A's and B's and C's for this epistemic two-armed bandit task, which is basically just like a team A's.

And then you build that out.

You know, you're writing in all the little sub-matrices of your A matrix.

That's why there's so many cells.

Like I said, it's like that's the longest part.

It's actually building things out.

The C vector, basically the reward function, which as Daphne was saying earlier, you're actually encoding in terms of relative log probabilities.

And then finally, you basically do those steps that we discussed during the presentation.

You just plug all your painstakingly generated A's and B's, hopefully not too painstaking, into your agent class.

You generate the two-armed bandit, the epistemic two-armed bandit environment, which is just the rules about how the world works, given the agent's actions.

And then you actually run this active inference loop, which is, as we discussed, just effectively running a loop over time, doing hidden state inference, policy inference, action sampling, and then stepping the environment to get the new observation.

And then at the end, I've just written this helper function that can basically plot the history of choices and beliefs.

So then you, at the very end, this is like the more fun experimental part.

You can mess with the parameters of the environment and also the parameters of the agent's model, and then start seeing how that changes behavior just by kind of iteratively running active inference simulations and plotting the resulting choice behavior and history of beliefs.

So that's the general, that's a little preview, I guess, of what we can do.

And we can also have a little sub module in here.

We're actually letting the agents update their beliefs about the A or the B matrix, which could be cool.

So, awesome.

Awesome.

Looks really exciting.

And on a final SPM note, in the SPM textbook and in experiments, sometimes there are these incredible grayscale matrices that summarize like multiple experimental factors across 100 participants.

And so it's really interesting to see how you show with also that black and white or grayscale matrix representation and how that provides a visual feel for some of these topics that we've been discussing.

And of course, the representation is formally linked with a matrix, but sometimes just saying, well, you have two options and there's 10 states in the world and the likelihoods look like this.

And so, instead of seeing that as like a spreadsheet with numbers, seeing them in the grayscale provides kind of a feel and it looks really nice.

And so it looks like an awesome session we'll have for dot two.

Yeah, totally.

That's interesting that you bring that up.

Like that's something I've just always been doing.

And I think it's very much because I learned all of that from reading those active inference and SPM papers.

So I very much just kind of borrowed that visualization technique from them.

And I kind of took it for granted.

But yeah, it is interesting.

It's clearly not the only way to go.

But I always just found it very intuitive to think of probability.

You just kind of can color it, use colors because the numbers are too specific.

It's the color, the grayscale, that visual aspect really like kind of just shows this thing is more likely than this thing.

Yeah.

Yep.

Cool.

All right.

Well, Yaka, Daphne and Connor, thanks a lot for this awesome session.

And we'll see you in a little bit more than a month for the dot two.

Great.

Thanks so much, Daniel.

And thank you, everyone.

Peace.

Take care.

Bye.

Thanks, everyone.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.

Bye.